

TOPIC 1:

PLAYLIST MANAGEMENT IN A MUSIC STREAMING APPLICATION

Name: M.Dilip Kumar

Reg No:192525038

Date:24/07/26

PLAYLIST MANAGEMENT USING DOUBLY LINKED LIST

Music Streaming Application

Data Structures Presentation

Name

Roll Number

Department

PROBLEM STATEMENT

Problem

A music streaming application must:

Add songs dynamically

Remove songs from any position

Reorder songs

Move to next or previous song

Maintain playlist efficiently

Requirements

Fast insertion

Fast deletion

Easy navigation

Dynamic memory usage

RECOMMENDED DATA STRUCTURE

Doubly Linked List

Each node stores:

- *Song information*
- *Previous pointer*
- *Next pointer*

Supports forward and backward traversal

No fixed size

Easy insertion and deletion

JUSTIFICATION

Why Doubly Linked List?

Advantages

O(1) insertion/deletion (with node reference)

Easy song reordering

Supports Previous and Next buttons

Dynamic memory allocation

Efficient playlist updates

CONCLUSION

Best choice for dynamic playlists

Flexible memory allocation

Efficient song navigation

Widely used in media players and streaming applications

TOPIC 2: DICTIONARY APPLICATION

RECOMMENDED DATA STRUCTURE: HASH TABLE

Fast Word Search Using Hash Table

Dictionary Application

Data Structures Presentation

PROBLEM STATEMENT

Problem

Dictionary must:

Check if a word exists

Search quickly

Store thousands of words

Minimize lookup time

Requirements

Fast searching

Efficient storage

Quick insertion

Quick deletion

RECOMMENDED DATA STRUCTURE

Hash Table

Stores key-value pairs

Uses a hash function

Converts word into an index

Directly accesses memory location

JUSTIFICATION

Why Hash Table?

Advantages

Extremely fast search

Direct access using hash function

Efficient insertion and deletion

Suitable for large dictionaries

CONCLUSION

Best data structure for dictionary applications

Provides constant-time search

Handles large databases efficiently

Used in spell checkers and search engines

TASK SCHEDULING SYSTEM

RECOMMENDED DATA STRUCTURE: PRIORITY QUEUE (HEAP)

Task Scheduling Using Priority Queue

Job Scheduling System

Data Structures Presentation

PROBLEM STATEMENT

Problem

Scheduler must:

Execute highest-priority task first

Add new tasks dynamically

Remove completed tasks

Maintain execution order

Requirements

Fast priority selection

Efficient scheduling

Dynamic updates

RECOMMENDED DATA STRUCTURE

Priority Queue (Binary Heap)

Stores tasks with priority

Highest priority served first

Binary Heap implementation

Automatically maintains priority order

JUSTIFICATION

Why Priority Queue?

Advantages

Fast retrieval of highest-priority task

Efficient insertion

Automatic priority ordering

Ideal for operating systems and schedulers

CONCLUSION

Best suited for task scheduling

Efficient priority management

Improves execution performance

Used in CPU scheduling, printers, and network routers